

УНИВЕРЗИТЕТ У БЕОГРАДУ

ФАКУЛТЕТ ОРГАНИЗАЦИОНИХ НАУКА

ЗАВРШНИ РАД

Утицај скупа алата на успешност и трошак
агената заснованих на великим језичким моделима
при решавању софтверских проблема

Ментор

др Андрија Петровић, Доцент

Студент

Алекса Љујић, 2021/0276

Београд, 2026. године

Посвета (по выбору)

АПСТРАКТ

Развој великих језичких модела довео је до појаве агентских система који, поред генерисања текста и кода, могу самостално да користе алате и делују у софтверском окружењу. У оваквим системима скуп доступних алата представља важан део интерфејса између језичког модела и окружења, али није у потпуности јасно у којој мери појединачни алати доприносе успешности решавања задатака и да ли њихова употреба оправдава додатни трошак. Циљ овог рада је испитивање утицаја скупа алата на успешност, трошак и способност локализације грешке код агента за решавање проблема у софтверском инжењерству, при фиксираном језичком моделу. У ту сврху развијен је сопствени агентски систем заснован на петљи позивања алата, са извршавањем команди у изолованим Docker контејнерима. Експеримент је спроведен на свих 300 задатака из скупа SWE-bench Lite, у факторском дизајну 2×2 , којим се независно испитује утицај алата за детерминистичку замену текста и групе алата за претрагу и читање датотека, док је алат за извршавање команди доступан у свим условима. Као примарне мере посматрају се стопа решених задатака, трошак по решеном задатку и стопа локализације грешке, док се додатно анализирају потрошња токена, број корака, време извршавања и коришћење појединачних алата. Поређење конфигурација заснива се на упареним статистичким поступцима, интервалима поверења и анализи неинфериорности. Посебна пажња посвећена је разликовању ефекта доступности алата од ефекта њиховог стварног коришћења. Резултати експеримента користе се за процену да ли мањи скуп алата може да обезбеди упоредиву успешност уз нижи трошак и да ли специјализовани алати пружају додатну корист при проналажењу датотеке која садржи грешку. На основу добијених резултата анализирају се компромиси између успешности и цене и издваја конфигурација која пружа најповољнији однос цене и перформанси.

Кључне речи: велики језички модели, агенти, агентски системи, позивање алата, софтверско инжењерство, SWE-bench Lite, аблациона анализа, локализација грешке, трошак, однос цене и перформанси.

САДРЖАЈ

1	Увод	1
1.1	Циљ рада	2
1.2	Методологија	2
1.3	Хипотезе	3
1.4	Структура рада	4
2	Релевантна литература	5
2.1	Темељи агената који користе алате	5
2.2	Бенчмаркови за генерисање кода и квалитет скупова података	5
2.3	Агентски системи за решавање проблема у софтверском инжењерству .	6
2.4	Коришћење алата, контекст и ефикасност	7
2.5	Методологија евалуације и статистичка поузданост	8
2.6	Празнина у литератури	8
3	Теоријске основе	10
3.1	Велики језички модели	10
3.1.1	Језичко моделовање	10
3.1.2	Трансформер архитектура	11
3.1.3	Генерисање и понашање модела у пракси	11
3.1.4	Ограничења	12
3.2	Агенти	12
3.2.1	Појам агента	12
3.2.2	Позивање алата	12
3.2.3	Петља закључивања и деловања	13
3.2.4	Архитектуре агентских система	14
3.3	Агенти за извршавање кода	14
3.3.1	Окружење и акције	14
3.3.2	Интерфејс између агента и рачунара	15
3.3.3	Изолација и поновљивост	15
3.4	Евалуациони оквир	16

3.4.1	Бенчмаркови за код	16
3.4.2	SWE-bench	16
3.4.3	Метрике	17
3.4.4	Статистичко поређење конфигурација	18
4	Методологија	20
4.1	Архитектура агента	20
4.2	Дизајн алата	21
4.3	Окружење за извршавање	23
4.4	Поступак евалуације	24
4.4.1	Пилот евалуација	24
4.4.2	Скуп података	24
4.4.3	Дизајн експеримента	25
4.4.4	Ток једног покретања	26
4.4.5	Мере	27
4.4.6	Правила искључивања	28
4.4.7	Статистичка анализа	28
5	Опис и резултати експеримента	30
5.1	Поставка експеримента	30
5.2	Резултати	30
5.2.1	Мера локализације	30
5.2.2	Провера хипотезе X2	32
5.3	Дискусија резултата	34
5.4	Ограничења	35
6	Закључак	38
	Употреба алата заснованих на вештачкој интелигенцији	39
	Литература	40

1 УВОД

Велики језички модели су у софтверском инжењерству најпре коришћени за допуњавање кода у развојном окружењу и као асистенти у облику разговора. У таквом начину рада контекст држи човек. Он бира који део кода ће моделу показати, преноси предлог решења у пројекат, покреће програм и проверава да ли измена заиста решава проблем. Модел даје предлог, али не делује у окружењу.

Последњих година тежиште се помера ка вештачкој интелигенцији као агенту. Агент је систем у коме језички модел у петљи сам бира наредни корак: чита датотеке репозиторијума, покреће команде, мења код и проверава резултат, све док задатак не буде решен. Истраживачки системи као што су SWE-agent (Yang, Jimenez, Wettig и др., 2024) и OpenHands (Wang и др., 2025), као и комерцијални алати засновани на истом принципу, показују да агенти могу да решавају стварне проблеме из репозиторијума отвореног кода. Скуп података SWE-bench (Jimenez и др., 2024) постао је стандардна мера напретка у овој области.

Успешност агента, међутим, не зависи само од модела. Систем који окружује модел (енг. *harness*) одређује које алате модел има на располагању, како су описани и какве повратне информације добија. Yang, Jimenez, Wettig и др. (2024) показују да дизајн интерфејса између агента и рачунара значајно утиче на резултат, иако је модел исти. Сваки алат ипак има и цену: његов опис се шаље моделу у сваком позиву, а резултати позива продужавају контекст, који се у наредним корацима поново обрађује и плаћа. Истовремено, Xia и др. (2025) показују да се конкурентни резултати могу постићи и знатно једноставнијим системом, што доводи у питање да ли је сложенији скуп алата заиста потребан. Недостаје систематско мерење изолованог доприноса појединачних алата, и то не само на успешност, него и на трошак.

Питање трошка има практичну тежину. Коришћење модела путем API-ја плаћа се по токenu, а агент у једном задатку прави десетине позива модела. Мали и јефтини моде-

ли су посебно занимљиви за примену, али је управо код њих отворено колико добро користе алате које им систем пружа. Овај рад се бави тим питањем на примеру малог модела и стварних задатака из SWE-bench скупа.

1.1 ЦИЉ РАДА

Општи циљ рада је да се измери како скуп алата доступних агенту утиче на успешност решавања задатака из софтверског инжењерства и на трошак тог решавања, при фиксираним језичком моделу.

Конкретни циљеви рада су:

- имплементирати сопствени агентски систем са скупом алата који се може конфигурисати и изолованим окружењем за извршавање;
- спровести аблацију скупа алата у факторском дизајну 2×2 : присуство алата за замену текста (`str_replace`) и присуство групе алата за претрагу и читање датотека, уз љуску (`bash`) као основни алат у свим условима;
- измерити стопу решених задатака званичном SWE-bench процедуром евалуације, трошак по решеном задатку, као и способност агента да пронађе датотеку у којој се налази грешка;
- утврдити која конфигурација алата пружа најбољи однос цене и перформанси, а која доноси предност при лоцирању грешке;
- утврдити у којој мери агент заиста користи алате који су му доступни, како би се ефекат доступности алата разликовао од ефекта његовог коришћења.

1.2 МЕТОДОЛОГИЈА

За потребе рада развијен је сопствени агентски систем у програмском језику Python. Агент ради у петљи позивања алата, а свака команда се извршава у изолованом Docker контејнеру без приступа мрежи, тако да агент не може да дође до званичне исправке проблема. Као модел се користи `gpt-5.4-nano`, са температуром 0 и искљученим

режимом проширеног закључивања. Модел је намерно фиксиран, јер је предмет рада скуп алата, а не поређење модела.

Експерименту је претходила пилот евалуација на мањем, синтетички направљеном скупу задатака опште намене (креирање и измена датотека, исправљање грешака, рад са тестовима). У пилоту је комбинација `bash` и `str_replace` остварила најнижи трошак и најкраће време извршавања уз високу успешност. Пошто је пилот рађен на малом броју задатака, са по једним покретањем и на вештачки направљеним задацима, тај налаз није довољан за закључак. Он је, међутим, мотивисао систематски експеримент на стварним задацима, описан у наставку.

Главни експеримент спроводи се на целом скупу SWE-bench Lite (300 задатака), у четири конфигурације алата, што даје 1200 покретања. Све конфигурације користе идентичан системски упит, који не помиње ниједан конкретан алат, тако да се између услова разликује само скуп доступних алата. Решења се оцењују званичним SWE-bench тестовима. Примарне мере су стопа решених задатака, трошак по решеном задатку и стопа локализације, тј. удео покретања у којима исправка агента мења бар једну датотеку коју мења и референтна исправка. Секундарне мере су трошак по покретању, број токена, број корака и време извршавања. Пошто сваки задатак пролази кроз све конфигурације, поређења су упарена. Мере, правила искључивања и конфигурација записани су пре покретања експеримента, а хипотезе и план статистичке анализе пре оцењивања решења и пре рачунања стопе локализације. Свако покретање бележи верзију кода којом је добијено. Детаљи су описани у поглављу 4.

1.3 ХИПОТЕЗЕ

Полазна теза рада је да комбинација љуске и алата за замену текста (`bash` и `str_replace`) пружа најбољи однос цене и перформанси, док пун скуп алата, који укључује и алате за претрагу и читање датотека, доноси предност пре свега при лоцирању грешке. Да би се ова теза могла проверити, рашчлањена је на три хипотезе:

- **X1:** Доступност алата за детерминистичку замену текста (`str_replace`) сма-

њује трошак по решеном задатку;

- **X2:** Доступност алата за претрагу и читање датотека повећава трошак по покретању, али повећава стопу локализације, тј. удео покретања у којима агент измени датотеку у којој се налази грешка;
- **X3:** Стопа решених задатака конфигурације са алатима `bash` и `str_replace` није нижа од стопе конфигурације са пуним скупом алата за више од пет процентних поена.

Хипотезе X1 и X2 проверавају се на главним ефектима факторског дизајна: ефекат алата `str_replace` добија се поређењем свих конфигурација са тим алатом и без њега, при оба нивоа другог фактора, а ефекат алата за претрагу и читање на исти начин. Хипотеза X3 је хипотеза неинфериорности и служи као контрола да уштеда из X1 није постигнута на рачун лошијих резултата. Ако се потврде све три хипотезе, из њих следи полазна теза: конфигурација са алатима `bash` и `str_replace` је најисплативији избор, а алати за претрагу и читање имају вредност при лоцирању грешке, али уз већи трошак. Разлике у ефекту између појединих врста задатака анализирају се експлоративно, без унапред формулисане хипотезе.

1.4 СТРУКТУРА РАДА

Рад је организован у шест поглавља. У другом поглављу дат је преглед литературе о агентима који користе алате, бенчмарковима за евалуацију генерисања кода и агентским системима за решавање проблема у репозиторијумима. Треће поглавље уводи теоријске основе: велике језичке моделе, појам агента и позивања алата, специфичности агената који раде над кодом и евалуациони оквир. Четврто поглавље описује методологију: архитектуру развијеног агента, дизајн алата и конфигурације које се пореде, окружење за извршавање и поступак евалуације. У петом поглављу приказани су поставка и резултати експеримента, њихова дискусија у односу на хипотезе и ограничења рада. Шесто поглавље сумира главне налазе и наводи правце даљег истраживања.

2 РЕЛЕВАНТНА ЛИТЕРАТУРА

Преглед литературе организован је тематски: најпре су приказани темељни радови о агентима који користе алате, затим бенчмаркови за евалуацију генерисања кода и њихова ограничења, агентски системи за решавање проблема у стварним репозиторијумима, радови о коришћењу алата и дугачког контекста, и на крају методолошки радови о статистички поузданој евалуацији.

2.1 ТЕМЕЉИ АГЕНАТА КОЈИ КОРИСТЕ АЛАТЕ

Парадигма ReAct (Yao и др., 2023) уводи наизменично смењивање корака закључивања и деловања, чиме језички модел може да планира, позива спољашње алате и прилагођава план на основу добијених опсервација. Ова петља представља основу већине савремених агентских система, укључујући и систем развијен у овом раду. Паралелно, Schick и др. (2023) показују да модел може самостално да научи када и како да позове алат (енг. *tool learning*), чиме се позивање алата успоставља као способност модела, а не само као спољашња надоградња.

2.2 БЕНЧМАРКОВИ ЗА ГЕНЕРИСАЊЕ КОДА И КВАЛИТЕТ СКУПОВА ПОДАТАКА

Рана евалуација језичких модела над кодом ослањала се на кратке, самосталне задатке, као што су HumanEval (Chen и др., 2021) и MBPP (Austin, Odena и др., 2021). Каснији бенчмаркови проширују обухват ка сложенијим позивима библиотека (Zhuo и др., 2024) и ка евалуацији отпорној на контаминацију тренинг података (Jain и др., 2024). J. Liu и др. (2023) показују да слаби тестови систематски прецењују перформансе модела, што је директно релевантно за метрику коришћену у овом раду.

SWE-bench (Jimenez и др., 2024) помера евалуацију ка стварним проблемима (енг. *issue*-

es) из GitHub репозиторијума, где модел мора да разуме постојећу кодну базу и произведе исправку (енг. *patch*), тј. скуп измена изворног кода који пролази задате тестове. Овај скуп података и његова евалуациона процедура представљају основу експерименталног дела рада. Проширења обухватају визуелне домене (Yang, Jimenez, Zhang и др., 2024) и више програмских језика (Zan и др., 2025). Истовремено, Aleithan и др. (2024) документују озбиљне проблеме квалитета у SWE-bench-у: у 32,67% успешних исправки решење је било садржано у самом опису проблема или коментарима (енг. *solution leakage*), а 31,08% прихваћених исправки сумњиво је због тестова који нису довољни да провере исправност. Након уклањања проблематичних задатака, стопа решених задатака система SWE-agent са моделом GPT-4 пала је са 12,47% на 3,97%. Ови налази се разматрају у поглављу о ограничењима.

2.3 АГЕНТСКИ СИСТЕМИ ЗА РЕШАВАЊЕ ПРОБЛЕМА У СОФТВЕРСКОМ ИНЖЕЊЕРСТВУ

Yang, Jimenez, Wettig и др. (2024) уводе појам интерфејса између агента и рачунара (енг. *Agent-Computer Interface*, ACI) и показују да дизајн скупа алата значајно утиче на успешност агента. Аблацијом на SWE-bench Lite скупу пореде различите начине претраге, прегледа и измене датотека. Посебно је релевантно поређење са агентом који има само љуску: исти модел решава 18,00% задатака са прилагођеним интерфејсом, а 11,00% само са љуском. Ова аблација представља најближи претходни рад за хипотезе о утицају скупа алата. OpenHands (Wang и др., 2025) је отворена платформа опште намене за софтверске агенте; у овом раду је ипак развијен сопствени систем, како би се обезбедила потпуна контрола над скупом алата који се пореди. HyperAgent (Phan и др., 2024) је систем са четири специјализована агента (планер, навигатор, уређивач кода и извршилац). Аутори спроводе аблацију улога агената и анализу утицаја појединих алата, а уз стопу решених задатака извештавају и просечан трошак по задатку, што га чини методолошки најближим овом раду. Међу осталим приступима издвајају се AutoCodeRover (Y. Zhang и др., 2024), који комбинује агента са техникама анализе програма, као и оквири за више агената попут AutoGen-a (Wu и др., 2023).

Посебно је важан рад Xia и др. (2025), који доводи у питање потребу за сложеним аген-

тима. Њихов приступ Agentless, заснован на трофазном процесу без агентске петље, постиже 32,00% (96 тачних исправки) на SWE-bench Lite скупу уз просечан трошак од 0,70 долара, надмашујући тадашње агенте отвореног кода. Аутори такође конструишу SWE-bench Lite-S уклањањем проблематичних инстанци (Xia и др., 2025). Овај рад полази од сродног питања, али из другог угла: уместо да уклони агентску петљу, задржава је и мери да ли специјализовани алати доносе стварну корист у односу на рад само са љуском.

Други правац истраживања фокусира се на тренирање модела за решавање проблема у репозиторијумима: SWE-Fixer (Xie и др., 2025), SWE-Gym (Pan и др., 2025), SWE-RL (Y. Wei и др., 2025) и SWE-smith (Yang и др., 2025). Ти приступи мењају сам модел, кроз додатно обучавање или учење поткрепљивањем, па захтевају податке за обучавање и знатне рачунарске ресурсе, а резултат важи за модел који је на тај начин добијен. Овај рад полази са друге стране: модел остаје непромењен, а мења се само оно што га окружује, односно скуп алата. Два приступа су допунска, јер се налази о дизајну алата могу применити на било који модел, укључујући и оне посебно обучене за решавање проблема у репозиторијумима.

2.4 КОРИШЋЕЊЕ АЛАТА, КОНТЕКСТ И ЕФИКАСНОСТ

ToolLLM (Qin и др., 2023) и Gorilla (Patil и др., 2023) проучавају способност модела да ефикасно користе велики број API-ја, док τ -bench (Yao и др., 2024) евалуира агенте у интеракцији са алатима и корисником. N. F. Liu и др. (2024) показују да модели слабије користе информације смештене у средину дугачког контекста, што објашњава зашто више контекста не значи нужно боље резултате и подржава претпоставку да мањи скуп алата може бити ефикаснији. У истом духу, Xiao и др. (2025) уклањају сувишне информације из трајекторије агента и тиме смањују број улазних токена за 39,9--59,7%, а укупан трошак за 21,1--35,9%, уз очуване перформансе.

2.5 МЕТОДОЛОГИЈА ЕВАЛУАЦИЈЕ И СТАТИСТИЧКА ПОУЗДАНОСТ

Bowman и Dahl (2021) разматрају системске проблеме бенчмаркова у обради природног језика. Card и др. (2020) показују да је велики број експеримената у овој области статистички подмоћан и дају смернице за процену потребне величине узорка. Drog и др. (2018) пружају практичан водич за избор статистичког теста, укључујући упарене тестове, а Reimers и Gurevych (2017) образлажу зашто треба извештавати дистрибуцију резултата уместо појединачне вредности. Ови радови усмеравају статистичку обраду резултата у поглављу 4.

2.6 ПРАЗНИНА У ЛИТЕРАТУРИ

Претходни радови показују да дизајн скупа алата утиче на успешност агента, али то питање углавном обрађују на један од два начина. Први је поређење целих система, као у радовима SWE-agent (Yang, Jimenez, Wettig и др., 2024), OpenHands (Wang и др., 2025) и Agentless (Xia и др., 2025), где се истовремено разликују и алати, и упити, и организација рада агента, па се допринос појединачних алата не може издвојити. Други је аблација у којој се компоненте уклањају једна по једна, као у радовима Yang, Jimenez, Wettig и др. (2024) и Phan и др. (2024), што показује да ли компонента помаже, али не и како се ефекти различитих компоненти међусобно односе. Факторски нацрт, у коме се две компоненте укрштају и посматра се и њихова интеракција, примењиван је и на друге делове агентског система, на пример на планирање и памћење ранијих покушаја (J. Zhang и др., 2026), али не и на скуп алата. Поред тога, евалуације се најчешће спровode са најјачим доступним моделима, док је мање познато колико успешно мали и јефтини модели решавају стварне задатке и колико им при томе помажу алати.

Овај рад доприноси у два правца. Главни допринос је примена факторског нацрта на скуп алата агента, односно систематско мерење ефекта две функционално различите групе: алата за измену кода (`str_replace`) и алата за претрагу и читање датотека. Групе се посматрају као два независна фактора у факторском дизајну 2×2 , уз љуску као заједничку основу, при чему системски упит и сви остали услови остају исти. Тиме се ефекат сваке групе мери одвојено, при оба нивоа друге групе, а могуће је уочити и

њихову међусобну интеракцију. Ефекат се не мери само на стопи решених задатака, већ и на трошку по решеном задатку и на способности агента да пронађе датотеку у којој се налази грешка.

Споредни допринос рада је увид у то колико успешно мали модел решава стварне проблеме из софтверског инжењерства. Експеримент се у целини спроводи са моделом `gpt-5.4-nano`, при температури 0 и без режима проширеног закључивања, на свих 300 задатака из скупа SWE-bench Lite. Резултати тако дају и основну меру могућности малог модела у агентском окружењу, као и одговор на питање колико му у томе помажу поједине групе алата.

3 ТЕОРИЈСКЕ ОСНОВЕ

Ово поглавље уводи појмове на којима почива остатак рада. Најпре су описани велики језички модели као основа система, затим појам агента и механизам позивања алата, који моделу омогућава да делује у окружењу. Након тога се разматрају специфичности агената који раде над изворним кодом, а поглавље се завршава описом евалуационог оквира: скупа података, метрика успешности и трошка, као и статистичких поступака за њихово поређење.

3.1 ВЕЛИКИ ЈЕЗИЧКИ МОДЕЛИ

3.1.1 Језичко моделовање

Језички модел дефинише расподелу вероватноћа над низовима токена, где је токен основна јединица текста (реч, део речи или симбол). Савремени модели су ауторегресивни: вероватноћа низа $x = (x_1, \dots, x_T)$ разлаже се на производ условних вероватноћа сваког токена у односу на претходне:

$$p_{\theta}(x) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}), \quad (1)$$

где θ означава параметре модела. Модел се тренира минимизацијом негативне лог-веродостојности над великим корпусом текста:

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}). \quad (2)$$

Дакле, основни задатак модела јесте предикција следећег токена. Сложеније способности, попут превођења, закључивања или писања кода, настају као последица тог једноставног циља примењеног на довољно велики модел и довољно велику количину података.

3.1.2 Трансформер архитектура

Готово сви савремени велики језички модели (енг. *Large Language Models*, LLM) засновани су на трансформер архитектури (Vaswani и др., 2017). Њен централни механизам је пажња (енг. *attention*), која сваком токену омогућава да директно приступи репрезентацијама свих претходних токена у контексту:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad (3)$$

где су Q , K и V матрице упита, кључева и вредности добијене линеарним пројекцијама улаза, а d_k димензија кључева. Пошто се пажња рачуна између свих парова токена, рачунска и меморијска сложеност расту квадратно са дужином контекста. Због тога модели имају ограничен контекстни прозор, а обрада дугачких улаза је скупља и спорија.

3.1.3 Генерисање и понашање модела у пракси

При генерисању, модел у сваком кораку израчунава логите z за све токене речника, који се претварају у вероватноће уз параметар температуре τ :

$$p(x_t = i \mid x_{<t}) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}. \quad (4)$$

Ниже вредности τ чине излаз детерминистичнијим, а више разноврснијим. Чак и при $\tau = 0$, излаз комерцијалних модела у пракси није потпуно детерминистички, што има директне последице по поновљивост експеримената.

Brown и др. (2020) показују да довољно велики модели могу да решавају нове задатке само на основу инструкција и примера у улазу (енг. *in-context learning*), без промене параметара. Додатно подешавање модела да прати инструкције, укључујући учење на основу људских повратних информација (Ouyang и др., 2022), чини моделе употребљивим као асистенте опште намене. Навођење модела да пре коначног одговора експлицитно изложи међукораке закључивања (енг. *chain-of-thought*) значајно побољ-

шава резултате на задацима који захтевају више корака (J. Wei и др., 2022).

3.1.4 Ограничења

Велики језички модели имају неколико ограничења која су релевантна за овај рад. Прво, знање модела је ограничено на податке из тренинга и не обухвата конкретну кодну базу над којом се ради, па модел мора да прикупи информације из окружења. Друго, модели могу да генеришу садржај који звучи уверљиво, али је нетачан (енг. *hallucination*) (Ji и др., 2023), нпр. да наведу непостојећу функцију или датотеку. Треће, иако контекстни прозори савремених модела обухватају стотине хиљада токена, модели не користе подједнако добро све делове контекста: N. F. Liu и др. (2024) показују да се информације у средини дугачког улаза користе знатно слабије од оних на почетку и крају. Четврто, трошак коришћења модела путем API-ја обрачунава се по броју улазних и излазних токена, па свака додатна информација у контексту има директну цену.

3.2 АГЕНТИ

3.2.1 Појам агента

У класичној литератури вештачке интелигенције, агент је систем који опажа своје окружење путем сензора и делује на њега путем актуатора, са циљем да оствари задати циљ (Russell & Norvig, 2020). У контексту великих језичких модела, агент је систем у коме језички модел понавља циклус: на основу тренутног стања одлучује коју акцију да предузме, акција се извршава у окружењу, а резултат се враћа моделу као нова опсервација. За разлику од једноставног упита, где модел даје један одговор, агент самостално одређује број и редослед корака потребних за решавање задатка.

3.2.2 Позивање алата

Језички модел сам по себи производи само текст. Да би деловао у окружењу, користи алате (енг. *tools*): функције које извршава спољашњи систем, као што су читање дато-

теке, претрага или покретање команде. Schick и др. (2023) показују да модел може да научи када и са којим аргументима да позове алат, а радови попут ToolLLM (Qin и др., 2023) и Gorilla (Patil и др., 2023) проширују ову способност на велики број стварних API-ја. Пошто је реч о научној способности, на понашање модела утиче и то колико су назив и облик понуђеног алата слични онима на којима је модел обучаван.

У савременим системима, сваки алат се описује називом, описом на природном језику и шемом параметара (најчешће у JSON Schema формату). Опис свих доступних алата укључује се у контекст модела, а модел, уместо слободног текста, може да генерише структурисани позив алата са аргументима. Систем који покреће агента (енг. *harness*) извршава позив и резултат враћа моделу. Важна последица је да *сваки алат заузима део контекста*: описи алата се шаљу у сваком позиву модела, а већи број алата повећава и број могућих избора које модел мора да разликује.

3.2.3 Петља закључивања и деловања

Парадигма ReAct (Yao и др., 2023) комбинује закључивање и деловање: модел наизменично генерише мисао (образложење следећег корака) и акцију (позив алата), након чега добија опсервацију. Формално, нека је q опис задатка. У кораку t контекст модела је

$$c_t = (q, r_1, a_1, o_1, \dots, r_{t-1}, a_{t-1}, o_{t-1}), \quad (5)$$

где су r_i мисли, a_i акције, а o_i опсервације. Модел генерише наредну мисао и акцију $(r_t, a_t) \sim p_\theta(\cdot \mid c_t)$, окружење враћа опсервацију $o_t = \text{Env}(a_t)$, и петља се наставља док модел не сигнализира крај или док се не достигне ограничење броја корака или трошка. Низ $(r_1, a_1, o_1, \dots, r_n, a_n, o_n)$ назива се трајекторија агента.

Из ове формулације следи важно својство трошка. Пошто се у сваком кораку моделу шаље цео досадашњи контекст, укупан број улазних токена у трајекторији дужине n износи

$$N_{\text{in}} = \sum_{t=1}^n |c_t|, \quad (6)$$

што расте приближно квадратно са бројем корака ако свака опсервација додаје сличан

број токена. Опширне опсервације (нпр. цео садржај великих датотека или дугачак излаз команди) и дугачки описи алата зато вишеструко повећавају трошак. Кеширање префикса упита (енг. *prompt caching*) код појединих провајдера смањује цену поновљених делова контекста, али не и њихов утицај на дужину контекста. Xiao и др. (2025) показују да се скраћивањем трајекторија може значајно смањити трошак агентских система.

3.2.4 Архитектуре агентских система

Агентски системи се разликују по степену аутономије. На једном крају су фиксни процеси у којима програмер унапред одређује кораке, а модел решава појединачне подзадатке; пример је Agentless (Xia и др., 2025), који задатак дели на локализацију, поправку и валидацију. На другом крају су аутономни агенти, у којима модел сам бира наредну акцију, као у SWE-agent (Yang, Jimenez, Wettig и др., 2024) и OpenHands (Wang и др., 2025). Постоје и системи са више агената који имају различите улоге и међусобно комуницирају, као AutoGen (Wu и др., 2023) и HyperAgent (Phan и др., 2024). Већа аутономија пружа флексибилност, али повећава број позива модела, трошак и варијабилност резултата.

3.3 АГЕНТИ ЗА ИЗВРШАВАЊЕ КОДА

3.3.1 Окружење и акције

Агенти за софтверско инжењерство раде над стварним репозиторијумом кода. Њихово окружење чине систем датотека, љуска (енг. *shell*) и програмски алати пројекта, као што су интерпретер, систем за управљање зависностима и тестови. Типичне акције таквих агената могу се груписати у неколико категорија:

- **навигација и претрага** — листање директоријума, претрага датотека по имену и садржају (нпр. `find`, `grep`);
- **читање** — приказ садржаја датотеке или њеног дела;

- **измена** — креирање датотека и измена постојећег кода, нпр. заменом тачно одређеног дела текста;
- **извршавање** — покретање произвољних команди, скрипти за репродукцију грешке и тестова.

Произвољна команда љуске је у принципу довољна за све наведене акције, па се поставља питање да ли специјализовани алати доносе стварну предност или само повећавају контекст. Управо ово питање је предмет хипотеза овог рада.

3.3.2 Интерфејс између агента и рачунара

Yang, Jimenez, Wettig и др. (2024) уводе појам интерфејса између агента и рачунара (ACI), по аналогији са корисничким интерфејсом за људе. Њихова теза је да језички модели имају другачије потребе од људи, па алати направљени за људе (нпр. интерактивни уређивачи текста) нису оптимални за агенте. Као принципе доброг интерфејса наводе једноставне акције са јасном документацијом, концизне и информативне повратне информације, као и заштитне механизме, нпр. проверу синтаксе након измене, који спречавају нагомилавање грешака. Аблацијом показују да дизајн интерфејса значајно утиче на успешност агента, иако је модел исти.

3.3.3 Изолација и поновљивост

Пошто агент извршава произвољне команде, окружење мора бити изоловано од рачунара на коме се систем покреће. Уобичајено решење су контејнери (нпр. Docker), који обезбеђују да сваки задатак почиње из истог, унапред дефинисаног стања, са инсталираним зависностима одговарајућих верзија. Изолација је истовремено безбедносни захтев и предуслов за поновљивост: разлике у окружењу не смеју да утичу на резултат поређења конфигурација.

3.4 ЕВАЛУАЦИОНИ ОКВИР

3.4.1 Бенчмаркови за код

Рани бенчмаркови, као што су HumanEval (Chen и др., 2021) и MBPP (Austin, Odena и др., 2021), састоје се од кратких, самосталних функција које модел генерише на основу описа, а исправност се проверава тестовима. За такве задатке уведена је метрика $\text{pass}@k$, вероватноћа да је бар једно од k генерисаних решења тачно. Ако се за задатак генерише $n \geq k$ решења, од којих је c тачних, непристрасна процена износи (Chen и др., 2021):

$$\text{pass}@k = \mathbb{E}_{\text{задаци}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]. \quad (7)$$

Такви задаци, међутим, не захтевају рад са постојећом кодном базом, што је суштина рада агената. Поред тога, J. Liu и др. (2023) показују да недовољни тестови доводе до прецењивања перформанси, јер нетачна решења пролазе проверу.

3.4.2 SWE-bench

SWE-bench (Jimenez и др., 2024) састоји се од 2294 задатка прикупљених из стварних проблема и одговарајућих захтева за спајање (енг. *pull requests*) у 12 популарних Python репозиторијума. Сваки задатак садржи опис проблема и стање репозиторијума пре исправке. Систем треба да произведе исправку у облику разлике у односу на изворни код. Исправка се сматра успешном ако након њене примене:

- прођу тестови који су пре исправке падали, а које је увела оригинална исправка (FAIL_TO_PASS), и
- и даље пролазе тестови који су пролазили и пре исправке (PASS_TO_PASS), чиме се проверава да исправка није покварила постојећу функционалност.

Поред пуног скупа, често се користи SWE-bench Lite, подскуп од 300 тест задатака који смањује трошак евалуације. Задаци су изабрани филтрирањем пуног скупа: уклоњени су задаци са сликама, спољашњим линковима и референцама на друге захтеве, задаци

чији опис има мање од 40 речи, као и задаци чија референтна исправка мења више од једне датотеке, има више од три измењена блока (енг. *hunks*) или креира и брише датотеке. Lite стога садржи самосталне, локализоване исправке грешака, што треба имати у виду при уопштавању резултата.

Приликом тумачења резултата треба имати у виду познате проблеме скупа. Aleithan и др. (2024) показују да је у значајном делу успешних решења исправка већ садржана у опису проблема, као и да су поједини тестови превише слаби да открију нетачне исправке. Такође, пошто су репозиторијуми јавно доступни, постоји ризик да су задаци или њихова решења били део тренинг података модела (енг. *data contamination*), што је мотивисало бенчмаркове попут LiveCodeBench (Jain и др., 2024).

3.4.3 Метрике

Нека је $\mathcal{D}$ скуп од N задатака, а $y_i \in \{0, 1\}$ индикатор да је задатак i решен. Основна метрика успешности је стопа решених задатака (енг. *resolve rate*):

$$R = \frac{1}{N} \sum_{i=1}^N y_i. \quad (8)$$

Трошак задатка i одређује се на основу броја улазних и излазних токена свих позива модела у трајекторији и цене по токenu коју одређује провајдер:

$$C_i = N_{\text{in}}^{(i)} \cdot p_{\text{in}} + N_{\text{out}}^{(i)} \cdot p_{\text{out}}, \quad (9)$$

где N_{in} следи из израза (6). Код провајдера који подржавају кеширање, улазни токени се додатно деле на кеширане и некеширане, са различитим ценама. Из трошка се изводе просечан трошак по задатку $\bar{C} = \frac{1}{N} \sum_i C_i$ и трошак по решеном задатку:

$$C_{\text{решен}} = \frac{\sum_{i=1}^N C_i}{\sum_{i=1}^N y_i}, \quad (10)$$

која обухвата и трошак неуспешних покушаја. Допунске метрике су број корака (позива модела) по задатку, укупан број токена и време извршавања.

Стопа решених задатака не показује да ли је агент бар пронашао место грешке. Зато се, по узору на Xia и др. (2025), посматра и локализација на нивоу датотеке. Нека је $F_i^{\text{агент}}$ скуп датотека које мења исправка агента, а $F_i^{\text{реф}}$ скуп датотека које мења референтна исправка задатка i . Стопа локализације је

$$L = \frac{1}{N} \sum_{i=1}^N \mathbb{I} \left[F_i^{\text{агент}} \cap F_i^{\text{реф}} \neq \emptyset \right]. \quad (11)$$

Пошто у скупу SWE-bench Lite референтна исправка увек мења тачно једну датотеку, $F_i^{\text{реф}}$ има један елемент, па метрика мери да ли је агент изменио управо ту датотеку. Ова метрика даје информацију и о задацима које агент није успео да реши, што је посебно корисно код слабијих модела, код којих је стопа решених задатака ниска.

Пошто се конфигурације разликују и по успешности и по трошку, поређење се врши у дводимензионалном простору. Конфигурација A доминира конфигурацијом B ако није ни скупља ни мање успешна, а бар по једном критеријуму је строго боља. Скуп недоминираних конфигурација чини Парето фронт, који приказује најбоље доступне компромисе између цене и перформанси.

3.4.4 Статистичко поређење конфигурација

Резултати агената подложни су случајности, како због стохастичности модела, тако и због коначног броја задатака. Reimers и Gurevych (2017) показују да поређење заснивано на једном покретању може довести до погрешних закључака, па је пожељно извештавати расподелу резултата кроз више покретања. Card и др. (2020) указују на то да су многи експерименти у обради природног језика статистички подмоћни, тј. да број примера није довољан да се поуздано открије разлика очекиване величине.

Када се две конфигурације евалуирају на истим задацима, опажања су упарена, па треба користити упарене тестове (Drog и др., 2018). За бинарни исход (решен/нерешен) одговарајући је Мекнемаров тест (McNemar, 1947), који разматра само неслагања: нека је b број задатака које решава само конфигурација A , а c број задатака које решава само конфигурација B . Под нултом хипотезом да су конфигурације подједнако успе-

шне, b има биномну расподелу $\text{Bin}(b + c, 0,5)$, а за довољно велико $b + c$ користи се апроксимација

$$\chi^2 = \frac{(b - c)^2}{b + c}. \quad (12)$$

За непрекидне метрике, као што је трошак по задатку, могу се користити Вилкоксонов тест ранга са знаком или пермутациони тестови. Интервали поверења за стопу решених задатака и разлике између конфигурација могу се проценити бутстреп методом (Efron & Tibshirani, 1993), поновљеним узорковањем задатака са враћањем. Исти поступак се примењује и на трошак по решеном задатку, који је количник збирова и нема вредност по појединачном задатку, па се за њега не може директно применити упарени тест.

Неуспех статистичког теста да открије разлику не значи да разлика не постоји. Када је циљ показати да нова конфигурација није битно лошија од референтне, користи се тест неинфериорности: унапред се одређује маргина δ , највећа разлика која се сматра прихватљивом, а неинфериорност се закључује ако доња граница интервала поверења за разлику $R_A - R_B$ лежи изнад $-\delta$. На тај начин се тврдња „конфигурације су подједнако успешне“ замењује проверљивом тврдњом „конфигурација A није лошија од конфигурације B за више од δ “.

4 МЕТОДОЛОГИЈА

Ово поглавље описује систем развијен за потребе рада и поступак којим је спроведен експеримент. Најпре је приказана архитектура агента, затим алати који му могу бити доступни и начин на који се бирају, окружење у коме се команде извршавају и, на крају, поступак евалуације: скуп података, дизајн експеримента, ток једног покретања, мере, правила искључивања и план статистичке анализе. Све вредности наведене у овом поглављу исте су у свим условима експеримента, осим скупа доступних алата.

4.1 АРХИТЕКТУРА АГЕНТА

Агент је реализован као петља позивања алата у духу парадигме ReAct (Ћао и др., 2023). Један корак петље састоји се од следећих радњи:

1. Модел добија историју поруке (системски упит, опис задатка и све досадашње позиве алата са резултатима) и JSON шеме активних алата.
2. Ако одговор модела не садржи позив алата, покретање се завршава (разлог завршетка `completed`).
3. У супротном се аргументи сваког позива валидирају, позив се извршава у изолованом окружењу, а резултат се враћа моделу као порука улоге `tool`.
4. Петља се понавља највише `max_turns` корака, након чега се покретање прекида (разлог завршетка `max_turns`).

Модел ништа не извршава директно, већ само предлаже позив алата, а систем одлучује да ли ће га и како извршити. Грешке су део разговора: неисправни аргументи, непознат алат или неуспешна команда не прекидају петљу, већ се моделу враћају као текстуална порука облика `ERROR: . . .`, на коју модел може да реагује. Излаз алата се тиме третира као део упита, па је јасноћа порука о грешкама део дизајна система.

Комуникација са моделом одвојена је од петље апстракцијом провајдера, која подржава OpenAI и Azure OpenAI API. Петља зна само за провајдерски неутралан облик одговора (текст, листа позива алата и потрошња токена), па се исти агент може користити са различитим провајдерима без измене логике.

Систем током рада бележи телеметрију: број улазних, излазних и токена закључивања, цену израчунату на основу ценовника модела, број позива модела, време проведено у позивима модела и у извршавању алата, као и број позива и грешака по алату. Провајдер повремено одбија исправан захтев због лажно позитивне класификације садржаја (`invalid_prompt`); такав захтев се понавља највише два пута, а свака друга грешка API-ја прекида покретање. Петља подржава и кооперативни сигнал за заустављање, који користи спољашње ограничење укупног трајања покретања.

Контекст се не сажима: историја поруке расте током целог покретања. Као што је показано у изразу (6), број улазних токена тада расте приближно квадратно са бројем корака, па је ограничавање величине излаза алата важан део контроле трошка.

4.2 ДИЗАЈН АЛАТА

Сваки алат је дефинисан називом, моделом аргумената за валидацију, JSON шемом која се шаље моделу и функцијом која га извршава. Алати су организовани у регистар из кога се за свако покретање може изабрати произвољан подскуп, што представља механизам аблације. У Табели 1 приказани су алати развијеног система.

Табела 1: Алати развијеног агентског система

Алат	Улога	Имплементација
bash	произвољна команда љуске; једини обавезан алат	docker exec
search	претрага садржаја регуларним изразом, највише 50 погодака, без .git и node_modules	grep у контејнеру
read_file	читање датотеке са бројевима линија и про- вером опсега	Python скрипта
find_file	претрага датотека по имену или шаблону, уз укупан број погодака	Python скрипта
str_replace	дословна замена текста; погодак мора бити јединствен, осим ако је изричито тражена за- мена свих појављивања	Python скрипта
run_tests	покретање тестова са структурираним изла- зом; <i>не користи се у експерименту</i>	Python скрипта

Дизајн алата прати принципе интерфејса између агента и рачунара (Yang, Jimenez, Wettig и др., 2024). Већина алата не додаје нову способност, јер се иста радња може обавити и командом љуске. Њихова вредност је у контроли излаза: резултат је ограни-
чен и предвидљив, па не заузима непотребно контекст. На пример, `search` ограничава
број погодака и изоставља директоријуме који нису део изворног кода, док би необра-
ђен `grep -r` над већим репозиторијумом могао у једном позиву да попуни велики
део контекста.

Алат `str_replace` је делимичан изузетак. Он решава задатак који се командама љу-
ске обавља непоуздано: измену вишелинијских блокова без проблема са означавањем
специјалних знакова и без тихе замене више појављивања. Без њега модел често мења
датотеку тако што је препише целу, што троши излазне токене и носи ризик од губитка
кода. Провера јединствености погодака је сигурносно својство алата: ако се тражени
текст појављује више пута, алат не мења ништа, већ враћа грешку са бројевима линија
свих погодака и саветом да се наведе шири контекст.

Поруке о грешкама и празним резултатима су изричите (нпр. `No matches` . или ознака да је излаз скраћен), јер их модел чита као упутство за наредни корак. Логика сложенија од једне команде љуске смештена је у скрипте које се у контејнер монтирају само за читање, а аргументи им се предају као JSON преко стандардног улаза, чиме се избегавају проблеми са означавањем знакова у команди.

4.3 ОКРУЖЕЊЕ ЗА ИЗВРШАВАЊЕ

Свако покретање добија сопствени, једнократни Docker контејнер. Коришћена слика садржи Python 3.12 и основне алате, али не и зависности пројекта над којим агент ради. Агент зато не може да покрене тестове пројекта, већ исправку прави на основу читања кода и описа проблема. Из истог разлога алат `run_tests` није део експеримента.

Јединствена слика за све репозиторијуме изабрана је због упоредивости услова и изводљивости експеримента: окружење је исто за сваки задатак, па разлике у зависностима не могу да утичу на поређење услова, а 1200 покретања остаје у оквиру расположивог времена и средстава. Припрема засебног окружења по задатку захтевала би вишеструко више простора и времена.

Последица те одлуке је да агент не може да покрене тестове пројекта нити да репродукује грешку, већ ради на основу читања кода и описа проблема. Успут се уклања и могућност да агент подешава исправку према постојећим тестовима, мада главну заштиту од доласка до званичног решења чине искључен приступ мрежи и уклоњена историја верзија, пошто скривени тестови којима се исправка оцењује ионако нису део радне копије.

Контејнер се покреће са ограниченим животним веком, тако да се уклања и ако систем неочекивано прекине рад, а команде прима преко `docker exec`. Систем датотека је трајан између позива, али стање љуске није: свака команда почиње у новој љусци, што је наведено и у системском упиту. Радна копија репозиторијума повезана је са контејнером као директоријум `/work` (енг. *bind mount*), па су измене агента одмах видљиве на рачунару домаћину, одакле се издваја исправка.

Контејнер ради са следећим ограничењима:

- без приступа мрежи (- - `network none`), па агент не може да преузме званичну исправку проблема;
- највише 2 GB меморије и 256 процеса;
- процеси раде под корисником домаћина, а не као `root`;
- скрипте алата доступне су само за читање;
- излаз команде ограничен је на 8000 знакова, уз изричиту ознаку скраћивања, а подразумевано време извршавања једне команде на 60 s.

На нивоу покретања, агент има највише 40 корака и 1800 s укупног времена.

4.4 ПОСТУПАК ЕВАЛУАЦИЈЕ

4.4.1 Пилот евалуација

Током развоја систем је провераван на 12 малих, синтетички направљених задатака (креирање и измена датотека, исправљање грешака, рефакторисање, рад са тестовима и CSV датотекама). Сваки задатак се извршава у новом контејнеру, а успех се утврђује провером стања система датотека након покретања, а не на основу онога што модел тврди да је урадио. Пошто су ови задаци коришћени за развој и подешавање система, а разликују се од стварних проблема из репозиторијума и по величини и по сложености, резултати пилота служе само као мотивација за главни експеримент и не представљају доказ за хипотезе.

4.4.2 Скуп података

Главни експеримент користи скуп SWE-bench Lite (Jimenez и др., 2024), у `test` подели, описан у поглављу 3, и то свих 300 задатака из 12 репозиторијума. Највише задатака потиче из репозиторијума `django` (114) и `sympy` (77). Пошто се користи цео

скуп, нема избора задатака од стране аутора, па ни пристрасности при избору. Сваки задатак обухвата репозиторијум, ознаку комита пре исправке (`base_commit`) и текст проблема. Скривени тестови (`FAIL_TO_PASS` и `PASS_TO_PASS`) агенту нису доступни.

4.4.3 Дизајн експеримента

Експеримент има факторски дизајн 2×2 са два фактора: присуство алата `str_replace` и присуство групе алата за претрагу и читање (`search`, `read_file` и `find_file`). Алат `bash` доступан је у свим условима. Комбинацијом нивоа добијају се четири услова приказана у Табели 2.

Табела 2: Услови експеримента

	без <code>str_replace</code>	са <code>str_replace</code>
без претраге и читања	А: <code>bash</code>	Б: <code>bash</code> , <code>str_replace</code>
са претрагом и читањем	В: <code>bash</code> , претрага	Г: сви алати

Сваки задатак пролази кроз сва четири услова, по једном, што даје 1200 покретања. Дизајн је упарен: задатак представља блок, па се услови пореде на истим задацима. Ефекат алата `str_replace` добија се поређењем услова А и Б, као и В и Г, а ефекат групе алата за претрагу и читање поређењем услова А и В, као и Б и Г.

Све остале променљиве су фиксирани и приказане у Табели 3. Посебно, сви услови користе исти системски упит, који не помиње ниједан конкретан алат нити обесхрабрује одређени начин измене датотека. На тај начин се мери ефекат доступности алата, а не ефекат алата у комбинацији са упутством да се он користи.

Табела 3: Фиксирани параметри експеримента

Параметар	Вредност
модел	gpt-5.4-nano
температура	0
режим проширеног закључивања	искључен
највећи број корака	40
највеће трајање покретања	1800 s
приступ мрежи	искључен
број понављања по услову	1
системски упит	идентичан, без помињања алата

4.4.4 Ток једног покретања

Једно покретање састоји се од четири фазе:

1. **Припрема.** Репозиторијум се из локалне копије поставља на стање комита пре исправке (`base_commit`). Историја верзија се затим брише и замењује једним почетним комитом без удаљеног репозиторијума, тако да агент не може да дође до касније, званичне исправке.
2. **Рад агента.** Агент добија текст проблема као задатак и ради над радном копијом у контејнеру.
3. **Издавање исправке.** Након завршетка, исправка се добија као разлика радне копије у односу на почетни комит. Бележе се и дијагностички подаци (изостала исправка, број измењених датотека и линија), а исправка се одмах уписује у званичном SWE-bench формату предвиђања.
4. **Оцењивање.** У одвојеној фази, званична SWE-bench процедура примењује исправку у Docker слици припремљеној за тај задатак и покреће скривене тестове. Задатак је решен ако пролазе сви `FAIL_TO_PASS` и `PASS_TO_PASS` тестови. Ако оцена не може да се добије (нпр. због проблема са Docker сликом), то се бележи засебно и не третира као неуспех агента.

За свако покретање чувају се ред са метрикама, цео разговор са моделом у читљивом облику, необрађене поруке које омогућавају тачно понављање анализе и детаљи евентуалне грешке. Уз сваки ред бележи се и ознака верзије кода система (`harness_sha`). Експеримент је започет пре него што су последње измене кода биле комитоване, па ова ознака указује на претходни комит. Измене које мењају понашање агента унете су пре почетка експеримента, а измене начињене током њега односе се искључиво на чување резултата и наставак прекинутог рада, тако да су сва покретања међусобно упоредива.

4.4.5 Мере

Мере су унапред подељене на три групе, како би се избегло накнадно тражење статистички значајних разлика међу великим бројем колона:

- **Примарне:** стопа решених задатака, трошак по решеном задатку (укупан трошак услова подељен бројем решених задатака у том услову) и стопа локализације, дефинисане у поглављу 3;
- **Секундарне:** трошак по покретању, број улазних и излазних токена, број корака и позива модела, укупно време, време у позивима модела и време извршавања алата;
- **Дијагностичке:** изостала исправка, разлог завршетка покретања, грешке алата, број позива по алату, број `FAIL_TO_PASS` и `PASS_TO_PASS` тестова који пролазе и величина исправке.

Трошак се рачуна према изразу из поглавља 3, на основу броја токена и јавног ценовника модела `gpt-5.4-nano` (0,20 долара по милиону улазних и 1,25 долара по милиону излазних токена). Стопа локализације рачуна се накнадно, поређењем датотека које мења исправка агента са датотеком коју мења референтна исправка; изостала исправка се рачуна као неуспешна локализација, по узору на `Xia` и др. (2025). Број позива по алату показује да ли је доступан алат заиста коришћен, чиме се ефекат доступности алата разликује од ефекта његовог коришћења.

4.4.6 Правила искључивања

Правила искључивања утврђена су пре увида у резултате:

- одбијање захтева због класификације садржаја (`invalid_prompt`) у примарној анализи сматра се исходом покретања, тј. нерешеним задатком, а стопа одбијања извештава се по услову;
- достизање највећег броја корака или највећег трајања такође је исход (нерешен задатак), а не квар;
- искључују се само покретања прекинута због грешке система или инфраструктуре. Јединица искључивања је задатак: ако је покретање искључено у једном услову, задатак се искључује из свих услова, како би број задатака по услову остао исти;
- анализа осетљивости понавља поређења само на задацима који су у сва четири услова завршени без грешке API-ја.

4.4.7 Статистичка анализа

Хипотезе X1 и X2 проверавају се на главним ефектима факторског дизајна, а хипотеза X3 поређењем услова Б и Г:

- **X1 (трошак по решеном задатку).** Упореди се трошак по решеном задатку услова са алатом `str_replace` (Б и Г) и без њега (А и В). Пошто је ова мера количник збирова, 95% интервал поверења за разлику процењује се бутстреп методом, поновљеним узорковањем задатака са враћањем, при чему се задатак узоркује истовремено у свим условима. Као допунски упарени тест, за сваки задатак рачуна се разлика просечног трошка по покретању са алатом и без њега и примењује Вилкоксонов тест ранга са знаком.
- **X2 (трошак и локализација).** Разлика у трошку по покретању између услова са групом алата за претрагу и читање (В и Г) и без ње (А и Б) тестира се Вилкоксоновим тестом над разликама по задатку. Разлика у стопи локализације процењује

се упареним бутстреп интервалом поверења, уз Мекнемаров тест за појединачна поређења А--В и Б--Г.

- **ХЗ (неинфериорност).** За разлику стопа решених задатака $R_B - R_G$ рачуна се 95% упарени бутстреп интервал поверења (Efron & Tibshirani, 1993). Неинфериорност се закључује ако је доња граница интервала изнад минус пет процентних поена.

За стопу решених задатака у свим појединачним поређењима услова користи се екзактни Мекнемаров тест (McNemar, 1947), у складу са препоруком да се за упарене податке користе упарени тестови (Drog и др., 2018). Пошто се тестира више хипотеза, p -вредности примарних тестова коригују се Холмовом методом. Уз p -вредности се увек извештавају величине ефекта и интервали поверења. Интеракција фактора, као и разлике у ефекту по репозиторијуму, величини референтне исправке и броју FA-IL_TO_PASS тестова, анализирају се експлоративно.

5 ОПИС И РЕЗУЛТАТИ ЕКСПЕРИМЕНТА

Ово поглавље приказује поставку спроведеног експеримента и добијене резултате. Пошто се оцењивање решености задатака званичним SWE-bench евалуатором спроводи одвојено и у тренутку писања још није завршено, у поглављу се проверава хипотеза X2, чије обе мере (трошак по покретању и стопа локализације) не захтевају покретање тестова. Провера хипотеза X1 и X3, које се ослањају на стопу решених задатака, спроводи се након оцењивања.

5.1 ПОСТАВКА ЕКСПЕРИМЕНТА

Поређена су четири услова добијена укрштањем два фактора: доступности алата `str_replace` за измену датотека и доступности групе алата за претрагу (`search`, `read_file` и `find_file`). Алат `bash` доступан је у свим условима, а услови су приказани у Табели 2.

Експеримент обухвата цео `test` подскуп збирке SWE-bench Lite (300 задатака) у сва четири услова, дакле $300 \times 4 = 1200$ покретања, са једним покретањем по услову. Агент ради у контејнеру без приступа мрежи, са највише 40 корака и ограничењем од 1800 секунди по задатку. Укупно утрошено време износи 15,3 сата, а укупан обрачунати трошак \$33,13, о чему се детаљније говори међу ограничењима рада.

5.2 РЕЗУЛТАТИ

5.2.1 Мера локализације

Пошто се поступак оцењивања званичним евалуатором спроводи одвојено, као примарна мера за хипотезу X2 користи се локализација: покретање се сматра локализованим ако скуп датотека које мења исправка агента и скуп датотека које мења рефе-

рентна исправка имају непразан пресек. Изостала исправка се по дефиницији сматра нелокализованом. Мера је укључена у план анализе пре израчунавања и не зависи од покретања тестова.

У Табели 4 приказани су резултати по условима, а у Табели 5 исходи покретања.

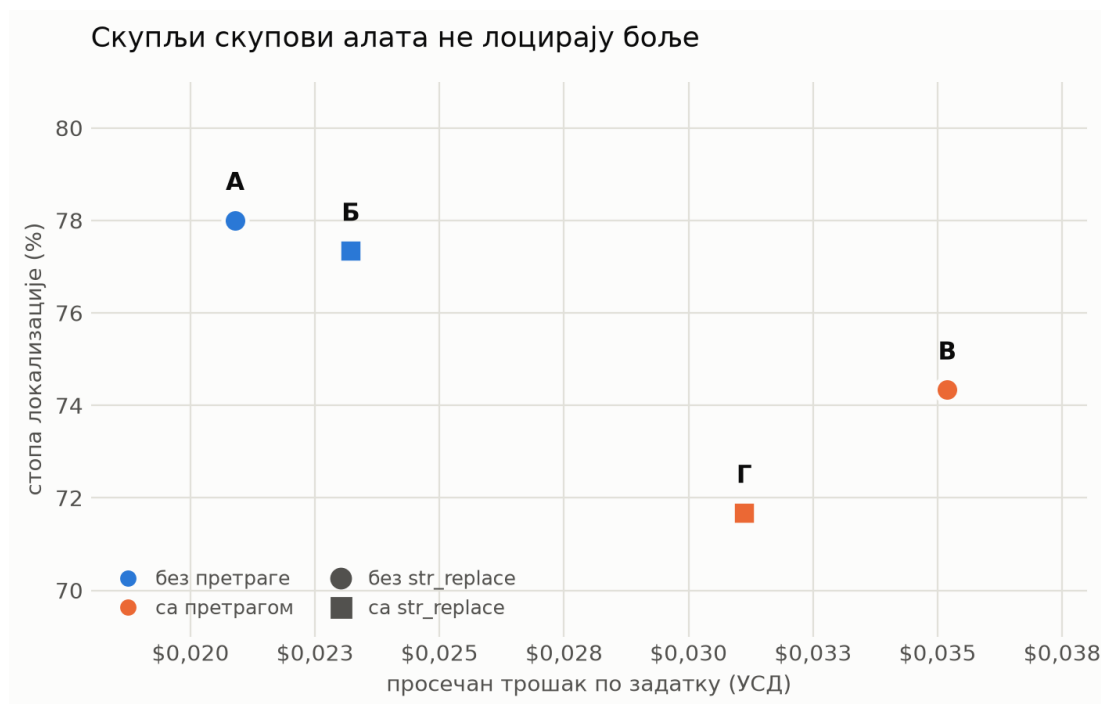
Табела 4: Резултати по условима (300 задатака по услову)

Услов	Алати	Локализација	Трошак/зад.	Корака	Токена
А	bash	78,0%	\$0,0209	16,0	96 550
Б	+ str_replace	77,3%	\$0,0232	15,8	108 896
В	+ претрага	74,3%	\$0,0352	23,9	168 070
Г	сви алати	71,7%	\$0,0311	22,4	149 454

Табела 5: Исходи покретања по условима

Услов	Алати	Изостала исправка	invalid_prompt	Исцрпљени кораци
А	bash	22	23	4
Б	+ str_replace	31	29	6
В	+ претрага	38	6	41
Г	сви алати	52	3	54

Однос између просечног трошка по задатку и стопе локализације приказан је на Слици 1. Услови са алатима за претрагу налазе се десно и ниже, што значи да су скупљи, а да притом не локализују боље.



Слика 1. Однос трошка по задатку и стопе локализације по условима

5.2.2 Провера хипотезе X2

Хипотеза X2 тврди да доступност групе алата за претрагу повећава трошак по покретању (X2a), али истовремено повећава стопу локализације (X2б). Према унапред утврђеном плану, X2 се сматра потврђеном само ако важе и X2a и X2б.

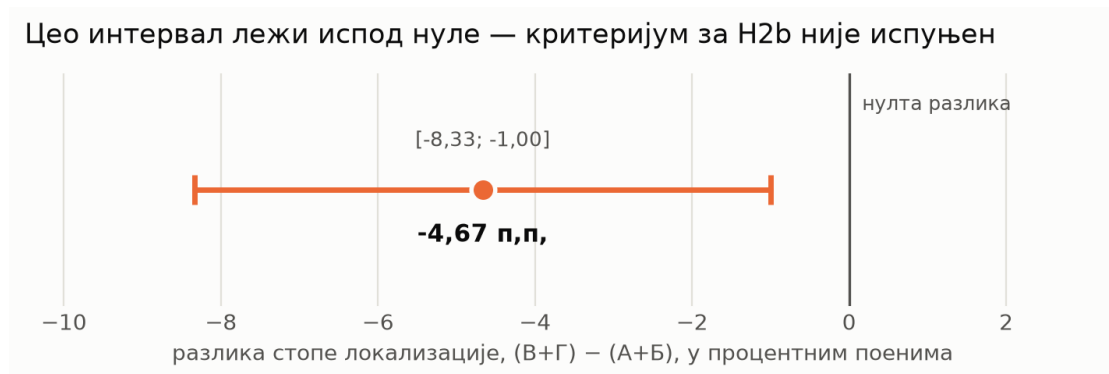
X2a — потврђена. Поређен је главни ефекат групе за претрагу, услови В и Г наспрам услова А и Б, над упареним разликама по задатку: за сваки задатак израчуната је разлика просечног трошка у условима са претрагом и просечног трошка у условима без ње. Медијана разлике износи $+\$0,0064$ (95% интервал поверења $[\$0,0050, \$0,0084]$), а Вилкоксонов тест упарених рангова даје $W = 6387$ и $p = 5,0 \cdot 10^{-27}$, уз величину ефекта $r_{rb} = +0,72$. Резултат опстаје после Холмове корекције.

Мера трошка рачуна се по ценовнику над свим улазним токенима, без попушта за кеширани улаз, што је описано међу ограничењима рада. Пошто услови са претрагом имају дуже путање, па и другачији удео кеширања, процењена величина ефекта у новчаним јединицама носи додатну несигурност. Зато се, као накнадна провера робусности изван унапред утврђеног плана, исти тест понавља над бројем токена по покретању,

величином на коју кеширање не утиче. Медијана разлике износи +32 886 токена (95% интервал поверења [+24 968, +42 255]), $W = 6036$, $p = 3,9 \cdot 10^{-28}$ и $r_{rb} = +0,73$, при чему услови са претрагом троше 1,55 пута више улазних токена. Смер и величина ефекта поклапају се са унапред утврђеним тестом, па закључак о хипотези Х2а не зависи од начина обрачуна кеширања.

Х2б — одбачена, и то у супротном смеру. Разлика стопе локализације између услова са претрагом и услова без ње износи $-4,67$ процентних поена, са упареним бутстреп интервалом поверења од 95% $[-8,33, -1,00]$ процентних поена (10 000 понављања, поновно узорковање задатака). Критеријум потврде био је да цео интервал лежи изнад нуле, а добијени интервал лежи цео испод нуле. Допунски Мекнемарови егзактни тестови дају за пар А--В $b = 31$, $c = 20$ и $p = 0,161$, а за пар Б--Г $b = 35$, $c = 18$ и $p = 0,027$. Ни један не опстаје после Холмове корекције примарне фамилије тестова, али оба имају исти смер као и интервални резултат.

Овај резултат приказан је и на Слици 2, где се види да цео интервал поверења лежи испод нуле.



Слика 2. Упарени бутстреп интервал поверења за разлику стопе локализације између услова са претрагом и услова без ње

Пошто Х2б није потврђена, хипотеза Х2 се одбацује. Штавише, оба њена дела показују ефекат супротан очекиваном: додавање алата за претрагу истовремено повећава трошак и смањује стопу локализације.

5.3 ДИСКУСИЈА РЕЗУЛТАТА

Резултат за хипотезу X_2 супротан је полазној претпоставци да наменски алати за претрагу помажу агенту да пронађе место грешке. Механизам се назива у Табелама 4 и 5: у условима са претрагом просечан број корака расте са приближно 16 на 22--24, а број покретања која исцрпе дозвољени број корака расте са 4--6 на 41--54. Уместо да скрате пут до одговарајуће датотеке, ти алати подстичу опширније претраживање, које троши буџет корака пре него што агент стигне до измене. Број изосталих исправки прати исти образац: 22 у услову А наспрам 52 у услову Г.

Овај налаз је у складу са резултатима N. F. Liu и др. (2024), по којима модели слабије користе информације у средини дугачког контекста, као и са тезом Xia и др. (2025) да сложенији систем не доноси нужно бољи резултат. Истовремено, он одступа од налаза Yang, Jimenez, Wettig и др. (2024), где је прилагођени интерфејс, укључујући и претрагу, побољшао успешност у односу на рад само са љуском. Разлика може потицати од величине модела, пошто је овде коришћен мали модел, коме дужи контекст и већи број могућих акција више отежавају него што помажу.

Разлика између услова А и Б није статистички разлучива на мери локализације ($b = 17$, $c = 15$, $p = 0,86$). То није доказ да алат `str_replace` нема ефекта, него последица тога што мера не може да га обухвати: тај алат делује у фази измене датотеке, након што је место грешке већ пронађено, док локализација мери искључиво проналажење. Вредност тог алата проверава се хипотезом X_1 , која се ослања на број решених задатака.

Од 300 задатака, на 180 су сва четири услова локализовала место грешке, а на 35 ни један. Преосталих 85 задатака, на којима се услови разликују, носе сав сигнал о утицају скупа алата. Овај однос је важан при тумачењу: удео задатака на којима се конфигурације уопште могу разликовати знатно је мањи од укупне величине збирке.

5.4 ОГРАНИЧЕЊА

Резултате рада треба тумачити у светлу следећих ограничења:

- **Један модел.** Експеримент је спроведен само са моделом `gpt-5.4-nano`, па се резултати не могу непосредно уопштити на јаче моделе. Пошто слабији модели вероватно имају више користи од помоћних алата, уочени ефекти представљају пре горњу него доњу границу.
- **Природа скупа SWE-bench Lite.** Скуп садржи искључиво самосталне исправке у једној датотеци са највише три измењена блока, што је тип измене у коме `str_replace` има највећу предност. Код измена кроз више датотека ефекат може бити мањи.
- **Квалитет скупа података.** Део задатака садржи решење у опису проблема, а део тестова није довољан да открије нетачне исправке (Aleithan и др., 2024). Постоји и ризик да су задаци били део података за тренирање модела. Резултати зато важе у оквиру бенчмарка.
- **Заштитни механизми провајдера.** Модел је коришћен преко Azure OpenAI сервиса, чији заштитни механизми (енг. *guardrails*) проверавају садржај сваког захтева. Пошто су захтеви садржали изворни код и излазе команди, класификатор је део њих означавао као неприхватљив садржај, иако то нису били, и враћао грешку `invalid_prompt`. Понављање захтева отклонило је део ових случајева, али не све, па одређени број задатака није могао да буде решен из разлога који не зависе ни од модела ни од скупа алата. Стопа таквих грешака извештава се по услову.
- **Недостајући подаци нису независни од услова.** Одбијања описана у претходној ставци нису равномерно распоређена: у условима без алата за претрагу било их је 52 (23 у услову А и 29 у услову Б), а у условима са претрагом свега девет (шест у услову В и три у услову Г). Разлог је то што агент без наменских алата више необрађеног кода пропушта кроз команде љуске. Недостајући подаци тиме корелирају са независном променљивом, па се њихова стопа извештава по услову, а налази се проверавају анализом осетљивости на задацима који су завршени

у сва четири услова.

- **Једно покретање по услову.** Није могуће проценити варијабилност резултата истог задатка у истом услову, а излаз модела није потпуно детерминистички ни при температури 0.
- **Агент ради „наслепо“.** Због јединствене слике окружења, описане у поглављу 4, агент не може да покрене тестове пројекта нити да репродукује грешку, него се ослања само на читање кода и опис проблема. То је значајно одступање од упоредивих система, попут SWE-agent (Yang, Jimenez, Wettig и др., 2024) и Agentless (Xia и др., 2025), који агенту омогућавају покретање тестова и проверу исправке, па је стопа решених задатака вероватно нижа него што би иначе била. Одлука важи подједнако у свим условима, тако да не утиче на поређење услова.
- **Усклађеност алата са обуком модела.** Позивање алата је научена способност (Schick и др., 2023), па оно зависи и од тога колико су назив и шема понуђеног алата слични онима које је модел видео током обуке. Алати развијени за овај рад не морају се поклапати са тим обрасцима, на шта указује и то што је модел у појединим покретањима позивао алате који у овом систему не постоје. Резултати стога важе за ову имплементацију алата, а другачије обликован интерфејс могао би дати другачији исход, што је у складу са налазом Yang, Jimenez, Wettig и др. (2024) да дизајн интерфејса битно утиче на успешност агента.
- **Мера локализације није мера исправности.** Локализација утврђује само да исправка агента дира бар једну датотеку коју дира и референтна исправка, не и да је измена исправна. Закључак о хипотези X2 стога важи за фазу проналажења места грешке, док се исправност проверава хипотезама X1 и X3, на основу резултата званичног евалуатора.
- **Мера трошка не обухвата кеширање.** Трошак се рачуна по ценовнику над свим улазним токенима, без попуста за кеширани улаз, а број кешираних токена није бележен током извођења, па се попуст не може накнадно применити. Улазни токени чине 94% обрачуна, а стварни трошак лежи између \$5,19 (уз потпуно кеширање) и \$33,13 (без кеширања), при чему се стварни рачун провајдера налазио близу доње границе. Пошто удео кеширања не мора бити једнак међу условима, поређење је поуздано по смеру ефекта, али процењена величина ефекта за хипо-

тезу X_2a у новчаним јединицама носи додатну несигурност. Провера над бројем токена, на који кеширање не утиче, даје исти закључак.

- **Тренутак утврђивања хипотеза.** Хипотезе и план статистичке анализе уписани су током извођења експеримента: услов А је био завршен у целости, услов Б делимично, а услови В и Г нису били покренути. Утврђивање је, међутим, извршено пре било каквог оцењивања решености и пре првог израчунавања локализације, тако да ни један исход који улази у проверу хипотеза није био познат у тренутку уписа.

6 ЗАКЉУЧАК

[Сажетак урађеног, главни налази, потврђене/оповргнуте хипотезе, правци за даља истраживања.]

ЛИТЕРАТУРА

- Aleithan, R., Xue, H., Mohajer, M. M., Nnorom, E., Uddin, G., & Wang, S. (2024). SWE-Bench+: Enhanced Coding Benchmark for LLMs. <https://arxiv.org/abs/2410.06992>
- Austin, J., Odena, A., и др. (2021). Program Synthesis with Large Language Models. <https://arxiv.org/abs/2108.07732>
- Bowman, S. R., & Dahl, G. E. (2021). What Will it Take to Fix Benchmarking in Natural Language Understanding? *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. <https://arxiv.org/abs/2104.02145>
- Brown, T. B., и др. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2005.14165>
- Card, D., Henderson, P., Khandelwal, U., Jia, R., Mahowald, K., & Jurafsky, D. (2020). With Little Power Comes Great Responsibility. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://arxiv.org/abs/2010.06595>
- Chen, M., и др. (2021). Evaluating Large Language Models Trained on Code. <https://arxiv.org/abs/2107.03374>
- Dror, R., Baumer, G., Shlomov, S., & Reichart, R. (2018). The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1383–1392. <https://aclanthology.org/P18-1128/>
- Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
- Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., & Stoica, I. (2024). LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. <https://arxiv.org/abs/2403.07974>
- Ji, Z., и др. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12). <https://arxiv.org/abs/2202.03629>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2310.06770>

- Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2023). Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2305.01210>
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173. <https://arxiv.org/abs/2307.03172>
- McNemar, Q. (1947). Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2), 153–157. <https://doi.org/10.1007/BF02295996>
- Ouyang, L., и др. (2022). Training Language Models to Follow Instructions with Human Feedback. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2203.02155>
- Pan, J., Wang, X., Neubig, G., Jaitly, N., Ji, H., Suhr, A., & Zhang, Y. (2025). Training Software Engineering Agents and Verifiers with SWE-Gym. *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2412.21139>
- Patil, S. G., Zhang, T., Wang, X., & Stoica, I. (2023). Gorilla: Large Language Model Connected with Massive APIs. <https://arxiv.org/abs/2305.15334>
- Phan, H. N., Nguyen, T. N., Nguyen, P. X., & Bui, N. D. Q. (2024). HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale. <https://arxiv.org/abs/2409.16299>
- Qin, Y., Liang, S., Ye, Y., и др. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. <https://arxiv.org/abs/2307.16789>
- Reimers, N., & Gurevych, I. (2017). Reporting Score Distributions Makes a Difference: Performance Study of LSTM-networks for Sequence Tagging. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://arxiv.org/abs/1707.09861>
- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4. издање). Pearson.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2302.04761>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/1706.03762>

- Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., и др. (2025). OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2407.16741>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2201.11903>
- Wei, Y., Duchenne, O., Copet, J., Carbonneaux, Q., Zhang, L., Fried, D., Synnaeve, G., Singh, R., & Wang, S. I. (2025). SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2502.18449>
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., и др. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. <https://arxiv.org/abs/2308.08155>
- Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2025). Agentless: Demystifying LLM-based Software Engineering Agents. *Proceedings of the ACM on Software Engineering (FSE)*. <https://arxiv.org/abs/2407.01489>
- Xiao, Y.-A., Gao, P., Peng, C., & Xiong, Y. (2025). Reducing Cost of LLM Agents with Trajectory Reduction. <https://arxiv.org/abs/2509.23586>
- Xie, C., Li, B., Gao, C., Du, H., Lam, W., Zou, D., & Chen, K. (2025). SWE-Fixer: Training Open-Source LLMs for Effective and Efficient GitHub Issue Resolution. *Findings of the Association for Computational Linguistics (ACL)*. <https://arxiv.org/abs/2501.05040>
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2405.15793>
- Yang, J., Jimenez, C. E., Zhang, A. L., Lieret, K., Yang, J., и др. (2024). SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains? <https://arxiv.org/abs/2410.03859>
- Yang, J., Lieret, K., Jimenez, C. E., Wettig, A., Khandpur, K., и др. (2025). SWE-smith: Scaling Data for Software Engineering Agents. <https://arxiv.org/abs/2504.21798>
- Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2024). τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. <https://arxiv.org/abs/2406.12045>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2210.03629>

- Zan, D., Huang, Z., Liu, W., Chen, H., Zhang, L., и др. (2025). Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. <https://arxiv.org/abs/2504.02605>
- Zhang, J., Zhang, Y., & Huang, Y. (2026). Coupling Planning with Episodic Memory in LLM Agents for Software Issue Resolution. <https://arxiv.org/abs/2608.06811>
- Zhang, Y., Ruan, H., Fan, Z., & Roychoudhury, A. (2024). AutoCodeRover: Autonomous Program Improvement. <https://arxiv.org/abs/2404.05427>
- Zhuo, T. Y., Vu, M. C., Chim, J., Hu, H., Yu, W., и др. (2024). BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2406.15877>